

GCLOCK (generalised clock)

- This is a mix between LFU (last frequently used) and LRU (last recently used) replacement policies.
- The reference counts are used to track references to cached objects.
- Each access request increments the reference count of the object.
- If the cache is full, the object to be removed is determined by decrementing the reference count for each object.
- After decrement, an object is replaced by the object in which reference count = 0 is found.
- This implementation tends to replace younger objects first.

ARC (adaptive replacement cache)

- ARC dynamically balances recency and frequency using a set of rules, and performs self-tuning.
- It keeps track of recently and frequently accessed data queues, along with the entries of recently and frequently removed data, also called ghost entries.
- These entries help the algorithm to expand or shrink the LRU or LFU, based on the usage.
- ARC enables substantial performance gains over commonly used modules.
- There is another variant of ARC — SARC or sequential prefetching in adaptive replacement cache — which is claimed to be better.

DeepBM

- This is a deep learning-based dynamic page replacement policy (https://people.eecs.berkeley.edu/~kubitron/courses/cs262a-F18/projects/reports/project16_report.pdf).
- Using deep learning, this algorithm learns from the past and dynamically adapts to the workload, which predicts the page to be evicted from the cache.

Cache policies

Cache policies determine how the cache operates in terms of writes to the storage.

- **Write around cache**
 - Writes to the storage first and skips the cache.
 - Here the advantage is when there is a large amount of write, the cache will not need to be overloaded with write I/O.
 - But there are two common problems — data could be stale in the cache, and reads could be slower as the new data will not be present in the cache.
- **Write through cache**
 - Write is performed in the system, cache, and actual storage.

- The advantage here is reads will be faster as the most recently written data is available in the cache.
- But write will be slower as it needs to wait until the write is successful in both the systems.
- **Write back cache**
 - Write operation takes place in the cache first, and is considered to be completed if the data is written to the cache.
 - From here, the data is copied back to the storage.
 - In this case, both the read and write could be faster.
 - But the greatest challenge is inconsistent writes, as there is a possibility of not being written to persistent storage.

Pros and cons of caching**Pros**

- Performance improvement is expected if the right cache algorithm is used.
- Can act as middleware when the connectivity is lost, allowing offline operations to sync once the system is online.

Cons

- Performance can be impacted if the cache ratio is low.
- Invalid or outdated information could result in misinformation, and there is a need to take special care that data in cache is not stale.

New caching techniques and algorithms are being continuously developed, but for now we have explored a few that can help better system performance. In conclusion, a quote to remember in the context of this article: *“There are only two hard things in computer science: cache invalidation and naming things.”*

— Phil Karlton 

References

- <https://coderanch.com/wiki/660295/Caching-Strategies>
- https://developers.redhat.com/sites/default/files/blog/2016/02/will-cohen-blog_graphics-02-300x300.png
- <https://www.youtube.com/watch?v=ccemOqDrc2I&list=LL&index=1>
- <https://www.techtarget.com/searchstorage/definition/cache>
- <https://hal.archives-ouvertes.fr/hal-01700364/documentIn>

By: Thangaselvi Arichandrapandian

The author works in a leading bank as AVP. She is a perennial learner and loves the quote: *“The more I learn, the more I realise how much I don't know.”* - Albert Einstein